



# BAHRIA UNIVERSITY, ISLAMABAD CAMPUS

Department of Computer Science

Course Code: CEN 323

Course Title: Computer Organization & Assembly Language (COAL)

## SEMESTER PROJECT REPORT

### Online Book Shop Management System

in 8086 Assembly Language using EMU8086

Submitted To:

Mr. Adnan Jelani

Lecturer, Department of Computer Science

Submitted By:

| Student Name    | Registration No. |
|-----------------|------------------|
| [MUHAMMAD UMER] | [01-135232-071]  |
| [IBRAHIM SAUD]  | [01-135232-059]  |

**Group Number:** [Group 10]

---

**Semester:** Spring 2026    —    **Submission Date:** [17 05 2026]

---

---

# Abstract

In the contemporary digital landscape, efficient management of retail inventories and customer interactions through software solutions has become an essential competency in computer science education. However, building such systems using low-level programming languages remains a challenging yet profoundly educational exercise. This project presents the design and implementation of an **Online Book Shop Management System** developed entirely in **8086 Assembly Language** using the **EMU8086** emulator, as a menu-driven DOS console application.

The system addresses the fundamental need for a structured, interactive retail simulation that exercises core computer organisation concepts. It implements a dual-access system supporting both user and administrative sessions, secured by credential-based authentication. Users may browse books by category, search by serial number, and complete purchases through an automated billing engine. The administrator role provides a consolidated inventory view. Key assembly programming concepts operationalised in this project include DOS interrupt services (INT 21H), BIOS screen interrupts (INT 10H), segmented memory management across the data, code, and stack segments, register-based arithmetic for billing calculations, stack-discipline procedure calls, string comparison routines, and conditional branching logic. The final system successfully executes on EMU8086 without errors, demonstrating that high-level functional requirements can be realised within the tight constraints of 16-bit assembly programming.

**Keywords:** 8086 Assembly Language, EMU8086, DOS Interrupts, INT 21H, Segmented Memory, Procedure Calls, Menu-Driven System, Billing System.

# Contents

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>Abstract</b> . . . . .                                 | <b>ii</b> |
| <b>1 Introduction</b> . . . . .                           | <b>1</b>  |
| 1.1 Background and Problem Statement . . . . .            | 1         |
| 1.2 Motivation . . . . .                                  | 1         |
| 1.3 Objectives . . . . .                                  | 2         |
| 1.4 Scope . . . . .                                       | 2         |
| 1.5 Importance of Assembly Language Programming . . . . . | 3         |
| 1.6 Why EMU8086 . . . . .                                 | 3         |
| <b>2 System Design</b> . . . . .                          | <b>5</b>  |
| 2.1 System Architecture Overview . . . . .                | 5         |
| 2.2 Memory Model and Segment Organisation . . . . .       | 5         |
| 2.3 Module Descriptions . . . . .                         | 6         |
| 2.4 Data Layout and Key Data Structures . . . . .         | 8         |
| 2.4.1 Static String Data . . . . .                        | 8         |
| 2.4.2 Buffered Input Buffers . . . . .                    | 8         |
| 2.4.3 Prices Array . . . . .                              | 8         |
| 2.5 Data Flow Diagram . . . . .                           | 9         |
| 2.6 Menu Flowchart . . . . .                              | 9         |
| 2.7 Design Decisions . . . . .                            | 9         |
| <b>3 Implementation Details</b> . . . . .                 | <b>12</b> |
| 3.1 Overview of Implementation . . . . .                  | 12        |
| 3.2 DOS Interrupt Services Used . . . . .                 | 12        |
| 3.3 Procedure-by-Procedure Implementation . . . . .       | 13        |
| 3.3.1 MAIN Procedure . . . . .                            | 13        |
| 3.3.2 USER_LOGIN and ADMIN_LOGIN Procedures . . . . .     | 14        |
| 3.3.3 STR_COMPARE Utility Procedure . . . . .             | 16        |
| 3.3.4 USER_MENU_PROC Procedure . . . . .                  | 17        |

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| 3.3.5    | CATEGORY_MENU Procedure . . . . .                     | 17        |
| 3.3.6    | SEARCH_BOOK Procedure . . . . .                       | 18        |
| 3.3.7    | BUY_BOOK Procedure and Billing Logic . . . . .        | 19        |
| 3.3.8    | PRINT_NUMBER Procedure . . . . .                      | 20        |
| 3.4      | Register Usage Summary . . . . .                      | 21        |
| 3.5      | Error Handling . . . . .                              | 21        |
| <b>4</b> | <b>CCA Mapping . . . . .</b>                          | <b>23</b> |
| 4.1      | CCA1 – Complex Engineering Problem Solving . . . . .  | 23        |
| 4.2      | CCA4 – Engineering Design Under Constraints . . . . . | 24        |
| 4.3      | CCA8 – Individual and Team Collaboration . . . . .    | 25        |
| <b>5</b> | <b>Testing &amp; Results . . . . .</b>                | <b>26</b> |
| 5.1      | Testing Strategy . . . . .                            | 26        |
| 5.2      | Test Cases . . . . .                                  | 26        |
| 5.3      | Screenshots . . . . .                                 | 28        |
| 5.3.1    | Welcome Screen . . . . .                              | 28        |
| 5.3.2    | Main Menu . . . . .                                   | 28        |
| 5.3.3    | User Login . . . . .                                  | 28        |
| 5.3.4    | Book Categories . . . . .                             | 29        |
| 5.3.5    | Billing Receipt . . . . .                             | 29        |
| 5.4      | Bug Fixes and Observations . . . . .                  | 29        |
| <b>6</b> | <b>Team Contributions . . . . .</b>                   | <b>32</b> |
| <b>7</b> | <b>Challenges &amp; Lessons Learned . . . . .</b>     | <b>33</b> |
| 7.1      | Challenges Encountered . . . . .                      | 33        |
| 7.1.1    | Register Management and Data Preservation . . . . .   | 33        |
| 7.1.2    | Debugging at the Assembly Level . . . . .             | 33        |
| 7.1.3    | Understanding DOS Interrupt Conventions . . . . .     | 33        |
| 7.1.4    | ASCII and Numeric Conversions . . . . .               | 34        |
| 7.1.5    | Memory Layout and Offset Arithmetic . . . . .         | 34        |
| 7.1.6    | Screen Management Limitations . . . . .               | 34        |
| 7.1.7    | Team Coordination . . . . .                           | 34        |
| 7.2      | Lessons Learned . . . . .                             | 34        |
| <b>8</b> | <b>Conclusion . . . . .</b>                           | <b>36</b> |
| 8.1      | Achievements . . . . .                                | 36        |

---

|                                                  |                               |           |
|--------------------------------------------------|-------------------------------|-----------|
| 8.2                                              | Learning Outcomes . . . . .   | 36        |
| 8.3                                              | Limitations . . . . .         | 37        |
| 8.4                                              | Future Improvements . . . . . | 37        |
| <b>References . . . . .</b>                      |                               | <b>39</b> |
| <b>A Complete Assembly Source Code . . . . .</b> |                               | <b>40</b> |
| <b>B Glossary of Assembly Terms . . . . .</b>    |                               | <b>54</b> |

# List of Figures

|     |                                                                   |    |
|-----|-------------------------------------------------------------------|----|
| 2.1 | High-Level System Architecture of the Online Book Shop . . . . .  | 5  |
| 2.2 | Data Flow Diagram of the Online Book Shop Management System . . . | 9  |
| 2.3 | Application Menu Navigation Flowchart . . . . .                   | 11 |
| 5.1 | Application Welcome Screen in EMU8086 . . . . .                   | 29 |
| 5.2 | Main Menu . . . . .                                               | 29 |
| 5.3 | Successful User Login . . . . .                                   | 30 |
| 5.4 | Programming Books Category View . . . . .                         | 30 |
| 5.5 | Sample Bill Receipt . . . . .                                     | 31 |

# List of Tables

|     |                                                          |    |
|-----|----------------------------------------------------------|----|
| 2.1 | Segment Organisation of the Application . . . . .        | 6  |
| 2.2 | Module Descriptions . . . . .                            | 7  |
| 2.3 | Buffered Input Buffer Structure (Function 0AH) . . . . . | 8  |
| 3.1 | DOS INT 21H Functions Used in the Application . . . . .  | 13 |
| 3.2 | BIOS INT 10H Functions Used . . . . .                    | 13 |
| 3.3 | Summary of Register Usage Across Procedures . . . . .    | 21 |
| 5.1 | Functional Test Cases and Results . . . . .              | 26 |
| 6.1 | Team Member Contributions . . . . .                      | 32 |
| 6.2 | Task Distribution by Activity . . . . .                  | 32 |
| B.1 | Glossary of Key Assembly Language Terms . . . . .        | 54 |

# 1. Introduction

## 1.1 Background and Problem Statement

Modern software development is dominated by high-level languages and frameworks that abstract away the underlying hardware. While this abstraction accelerates development productivity, it simultaneously conceals the foundational mechanisms upon which all computation rests. Students of computer science must, at some stage, engage directly with the hardware-software interface to develop a genuine understanding of processor architecture, memory organisation, instruction execution, and I/O management.

The **Online Book Shop Management System** was conceived to bridge this gap. Rather than merely studying assembly language through isolated programs, the project demands the integration of multiple assembly programming constructs into a cohesive, functional application. The problem being solved is twofold: *technically*, to implement a fully navigable, feature-complete book shop management application using only 8086 instructions and DOS system calls; and *educationally*, to consolidate understanding of data segments, code segments, stack operations, interrupts, procedures, loops, and conditional logic in a single, meaningful project.

## 1.2 Motivation

Several motivating factors drove the selection of this project topic:

- **Real-World Context:** A book shop management system is immediately relatable, making abstract concepts such as arrays, billing arithmetic, and string comparison tangible.
- **Educational Depth:** Implementing login authentication, category menus, search, and billing at the assembly level requires mastery of register management, the stack, and interrupt-driven I/O — all core examination and professional competencies.
- **Constraint-Driven Design:** The strict limitations of the 8086 architecture (16-bit registers, segmented memory, no native string libraries) force students to

engage creatively with hardware constraints, an invaluable skill for embedded systems and systems programming careers.

## 1.3 Objectives

The primary objectives of this project are as follows:

1. To design and implement a fully functional, menu-driven book shop management system in 8086 Assembly Language.
2. To apply DOS interrupt services (particularly INT 21H functions) for all console input/output operations.
3. To implement a credential-based dual-access (User/Admin) login system using string comparison at the assembly level.
4. To implement book catalogue browsing by category, book search by serial number, and a purchase and billing subsystem.
5. To demonstrate correct use of the data segment (static strings and arrays), the code segment (procedures), and the stack segment (procedure linkage).
6. To apply disciplined register usage, avoiding data corruption across procedure boundaries through stack push/pop discipline.
7. To produce a well-documented, readable, and maintainable assembly source file.

## 1.4 Scope

The scope of this project is bounded by the following parameters:

- The application is a single-process, single-user, text-mode DOS console program.
- It runs within the **EMU8086** software emulator and targets the Intel 8086 processor instruction set.
- The book inventory consists of **six** books across three categories (Programming, Fiction, Science), stored as static data in the data segment.
- User and administrator credentials are hard-coded in the data segment for demonstration purposes; no file I/O or persistent storage is implemented.
- Billing calculations support single-item purchases with quantities between 1 and

9, using 16-bit unsigned multiplication.

- The administrator role provides inventory viewing only; no runtime inventory editing or stock management is implemented within scope.

## 1.5 Importance of Assembly Language Programming

Assembly language occupies a unique position in computer science: it is the lowest-level human-readable representation of machine instructions. Understanding assembly language provides several critical benefits:

- **Hardware Transparency:** Every operation corresponds directly to a processor instruction; there is no compiler-mediated abstraction. This makes the programmer acutely aware of how the CPU fetches, decodes, and executes instructions.
- **Memory Awareness:** The programmer manages memory explicitly, declaring data in named segments and accessing it through segment:offset addressing. This cultivates an intuitive understanding of virtual memory, pointers, and memory layout — concepts critical to systems programming.
- **Performance Optimisation:** Expert assembly programmers can write code that outperforms compiled high-level code in performance-critical sections, a skill still valued in embedded systems, device drivers, and real-time applications.
- **Security Research:** Reverse engineering, exploit development, and vulnerability analysis are deeply dependent on assembly language literacy.

## 1.6 Why EMU8086

EMU8086 was selected as the development and execution platform for the following reasons:

- **Integrated Development Environment:** EMU8086 provides an assembler, linker, and emulator within a single application, eliminating complex toolchain configuration.
- **Visual Debugging:** The emulator offers real-time register views, flag states, stack contents, and memory dumps, making it an ideal pedagogical tool for

observing program execution.

- **DOS Compatibility:** It faithfully emulates the DOS environment, supporting the full INT 21H function set and the BIOS INT 10H video services used in this project.
- **Educational Standard:** EMU8086 is widely adopted in university assembly language courses globally, ensuring ample documentation, community support, and instructor familiarity.

## 2. System Design

### 2.1 System Architecture Overview

The Online Book Shop Management System follows a **modular, procedure-based architecture** within a single-segment small memory model (`.MODEL SMALL`). The application is structured around a central main loop that dispatches control to independent procedure modules based on user input. Each procedure is self-contained: it uses the stack to preserve registers, performs its function, and returns cleanly, with no global side effects other than the shared `total` variable used during billing.

Figure 2.1 illustrates the high-level architectural organisation of the system.

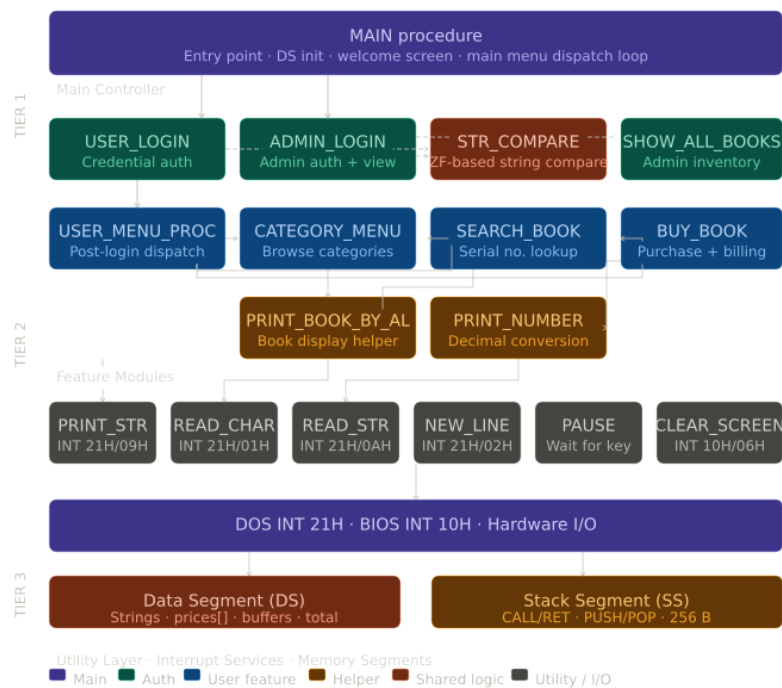


Figure 2.1: High-Level System Architecture of the Online Book Shop

### 2.2 Memory Model and Segment Organisation

The 8086 processor employs a **segmented memory model** in which the address space is divided into 64 KB segments addressed by dedicated segment registers. This

project uses the `.MODEL SMALL` directive, which places the data segment and the code segment each within a single 64 KB segment, with both accessible without segment override prefixes when `DS` and `CS` are correctly initialised.

Table 2.1: Segment Organisation of the Application

| Segment       | Register | Contents / Purpose                                                                                                                                                                                                                       |
|---------------|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Data Segment  | DS       | All static strings (menus, prompts, messages), input buffers ( <code>buf_user</code> , <code>buf_pass</code> ), the <code>prices</code> word array, and the <code>total</code> word variable.                                            |
| Code Segment  | CS       | All executable instructions: the <code>MAIN</code> procedure, all feature procedures ( <code>USER_LOGIN</code> , <code>ADMIN_LOGIN</code> , etc.), and all utility procedures ( <code>PRINT_STR</code> , <code>READ_CHAR</code> , etc.). |
| Stack Segment | SS       | A 256-byte (100H) stack used for procedure <code>CALL/RET</code> return address management and register preservation via <code>PUSH/POP</code> instructions.                                                                             |

At program entry, the `MAIN` procedure initialises the data segment register:

```

1  MOV AX, @DATA          ; Load data segment address
2  MOV DS, AX             ; Point DS to the data segment

```

Listing 2.1: Data Segment Initialisation at Program Entry

This is a mandatory step in `.MODEL SMALL` programs because the `DS` register is not automatically set by the operating system loader in DOS.

## 2.3 Module Descriptions

The application is decomposed into the following logical modules, each implemented as one or more assembly procedures:

Table 2.2: Module Descriptions

| Module          | Procedure(s)            |
|-----------------|-------------------------|
| Main Controller | MAIN                    |
| Authentication  | USER_LOGIN, ADMIN_LOGIN |
| User Interface  | USER_MENU_PROC          |

## 2.4 Data Layout and Key Data Structures

### 2.4.1 Static String Data

All user-visible text is declared as DB (define byte) strings terminated by the \$ character, the convention required by INT 21H function 09H (print string). Each string is a named label in the data segment, accessed via the LEA DX, *label* instruction.

### 2.4.2 Buffered Input Buffers

User credential input uses INT 21H function 0AH (buffered keyboard input), which writes to a structure of the form:

Table 2.3: Buffered Input Buffer Structure (Function 0AH)

| Byte Offset | Content      | Description                                                 |
|-------------|--------------|-------------------------------------------------------------|
| 0           | 10           | Maximum number of characters to accept                      |
| 1           | (set by DOS) | Actual number of characters entered                         |
| 2–11        | Input data   | Characters typed by user, followed by 0Dh (carriage return) |

In the source, `buf_user` and `buf_pass` are each declared as:

```

1  buf_user  db 10, 0, 10 dup('$')    ; max 9 chars + terminator
2  buf_pass  db 10, 0, 10 dup('$')
```

Listing 2.2: Input Buffer Declaration

### 2.4.3 Prices Array

Book prices are stored as a parallel word array:

```

1  prices    dw 250, 400, 500, 300, 450, 350
```

Listing 2.3: Prices Array Declaration

Indexed access uses the book serial number (converted to a zero-based index and scaled by 2 for word addressing) via the SHL instruction.

## 2.5 Data Flow Diagram

Figure 2.2 illustrates the Data Flow Diagram (DFD Level 0 and Level 1) of the Online Book Shop Management System, showing the interaction between the User/Admin, Login Module, Menu Modules, Billing Module, and Data Store.

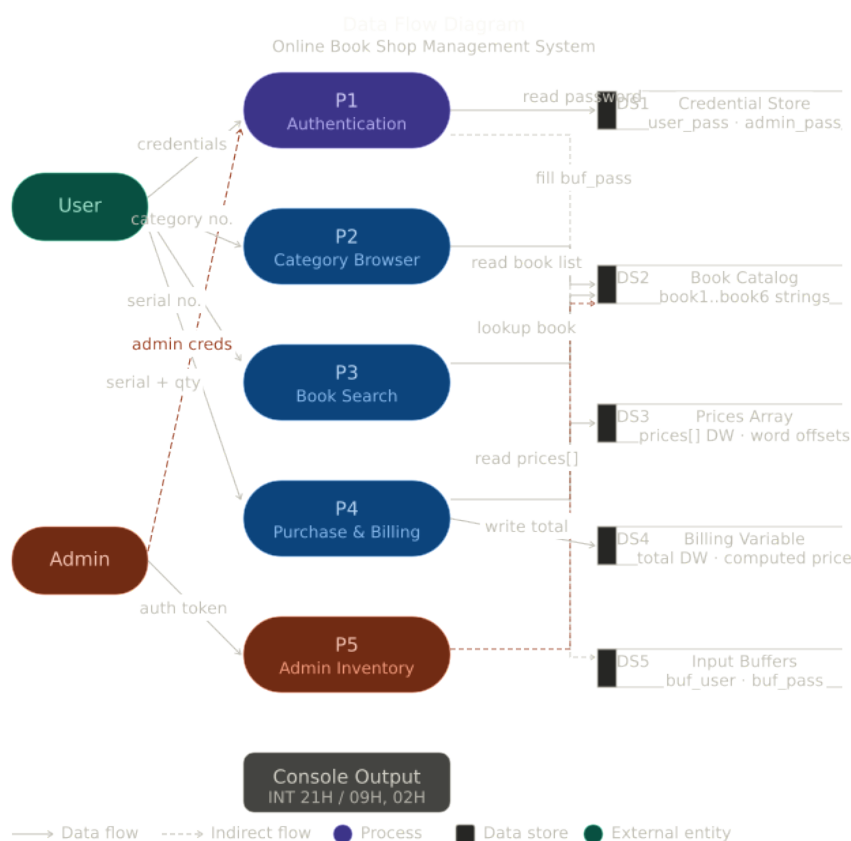


Figure 2.2: Data Flow Diagram of the Online Book Shop Management System

## 2.6 Menu Flowchart

Figure 2.3 illustrates the navigation structure of the Online Book Shop Management System, including the Welcome Screen, Main Menu, User Login, Admin Login, User Menu, Categories, Search, Buy Book, Logout, and Exit functionalities.

## 2.7 Design Decisions

1. **Hard-coded Credentials:** Due to the limited capabilities of the 8086 Assembly environment and the absence of database or file handling support in this project

scope, user and admin credentials are stored directly in the data segment as \$-terminated strings. This simplified approach is suitable for demonstrating authentication logic in an educational environment.

2. **Single-character Menu Input:** The system uses DOS interrupt INT 21H function 01H for menu input handling. Since menu options are represented by single ASCII characters, this method provides a simpler and faster implementation compared to buffered keyboard input methods.
3. **Parallel Data Structures:** Book titles and their corresponding prices are maintained using separate but parallel data structures. Book information is stored as individual string labels, while prices are stored in a word array named `prices`. This design simplifies memory access and indexing operations in Assembly Language.
4. **BIOS-based Screen Clearing:** Screen clearing functionality is implemented using BIOS interrupt INT 10H function 06H. This method scrolls the screen window upward to clear the display efficiently and ensures compatibility with both EMU8086 and DOS-based systems.
5. **Modular Procedure Design:** The application is divided into multiple procedures such as login handling, category display, search functionality, billing, and utility functions. This modular structure improves readability, simplifies debugging, and enhances code maintainability.
6. **Menu-driven Program Flow:** A hierarchical menu-based structure is used throughout the application. This approach allows users to navigate easily between different modules while maintaining clear and organized program flow control.

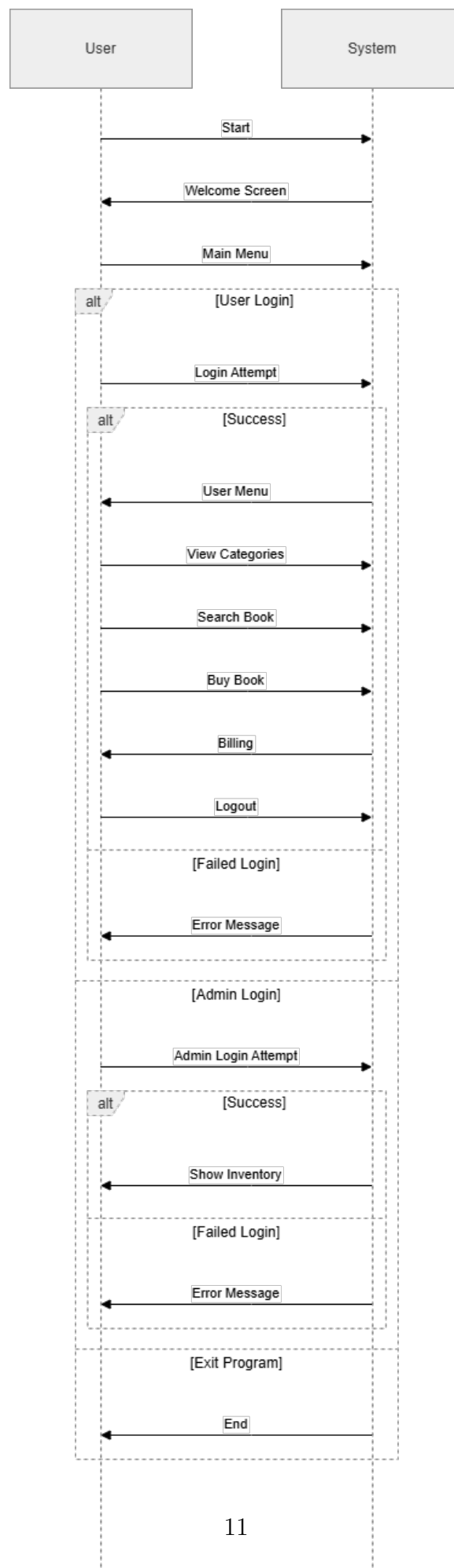


Figure 2.3: Application Menu Navigation Flowchart

## 3. Implementation Details

### 3.1 Overview of Implementation

The entire application is contained in a single `.asm` source file, assembled using EMU8086's built-in assembler. The `.MODEL SMALL` directive generates a COM-like executable with separate but compact data and code segments. The `.STACK 100H` directive allocates 256 bytes of stack space, sufficient for the procedure call depth of this application (maximum nesting: `MAIN`  $\rightarrow$  `USER_LOGIN`  $\rightarrow$  `USER_MENU_PROC`  $\rightarrow$  `BUY_BOOK`  $\rightarrow$  `PRINT_NUMBER`, approximately 5 levels deep, each using at most 8 bytes of stack for pushed registers and the return address).

### 3.2 DOS Interrupt Services Used

All input/output in the application is performed through **DOS interrupt** `INT 21H`. Table 3.1 summarises every function used.

Table 3.1: DOS INT 21H Functions Used in the Application

| AH  | Function Name           | Description                                                     | Used In                      |
|-----|-------------------------|-----------------------------------------------------------------|------------------------------|
| 01H | Character Input w/ Echo | Reads one character from stdin; echoes it; returns ASCII in AL. | Menu selection, search, buy  |
| 02H | Character Output        | Outputs the character in DL to stdout.                          | Billing digit print, newline |
| 09H | String Output           | Prints a \$-terminated string pointed to by DS:DX.              | All string display           |
| 0AH | Buffered String Input   | Reads a string into a buffer at DS:DX; max length in byte 0.    | Username/password input      |
| 4CH | Terminate Program       | Exits the program and returns control to DOS.                   | GO_EXIT in MAIN              |

Additionally, **BIOS interrupt** INT 10H is used for screen management:

Table 3.2: BIOS INT 10H Functions Used

| AH  | Function            | Description                       | Parameters                       |
|-----|---------------------|-----------------------------------|----------------------------------|
| 06H | Scroll Up Window    | Clears the screen by scrolling up | AL=0 rows (all), BH=attr, CX/DX= |
| 02H | Set Cursor Position | Repositions cursor to top-left    | BH=page 0, DX=0000H              |

## 3.3 Procedure-by-Procedure Implementation

### 3.3.1 MAIN Procedure

The **MAIN** procedure is the application entry point. It initialises the data segment, displays the welcome screen, and enters an infinite dispatch loop that maps user input characters to the corresponding login or exit procedures.

```

1  MAIN PROC
2      MOV AX, @DATA                ; Load data segment selector
3      MOV DS, AX                  ; Initialise DS
4
5      CALL CLEAR_SCREEN            ; Clear display
6      LEA DX, welcome_msg         ; Point DX to welcome banner
7      CALL PRINT_STR              ; Display it
8      CALL READ_CHAR              ; Wait for any keypress
9
10 MAIN_LOOP:
11     CALL CLEAR_SCREEN
12     LEA DX, main_menu
13     CALL PRINT_STR
14     CALL READ_CHAR               ; AL = '1', '2', or '3'
15
16     CMP AL, '1'
17     JE  GO_USER                 ; Jump if User Login selected
18     CMP AL, '2'
19     JE  GO_ADMIN               ; Jump if Admin Login selected
20     CMP AL, '3'
21     JE  GO_EXIT               ; Jump if Exit selected
22
23     LEA DX, err_choice         ; Invalid input handler
24     CALL PRINT_STR
25     CALL PAUSE
26     JMP MAIN_LOOP             ; Re-display menu
27
28 GO_EXIT:
29     LEA DX, bye_msg
30     CALL PRINT_STR
31     MOV AH, 4Ch                ; DOS terminate function
32     INT 21h
33 MAIN ENDP

```

Listing 3.1: MAIN Procedure – Segment Init and Dispatch Loop

**Registers used:** AX (segment init, input), DX (string pointer), DS (segment base).

### 3.3.2 USER\_LOGIN and ADMIN\_LOGIN Procedures

The login procedures accept username and password inputs using buffered string input (INT 21H / 0AH). The entered password is then compared byte-by-byte against the

hard-coded credential string using the `STR_COMPARE` utility. On success, control is delegated to the user menu or admin view respectively; on failure, an error message is displayed and the procedure returns.

```

1  USER_LOGIN PROC
2      CALL CLEAR_SCREEN
3      LEA DX, prompt_user
4      CALL PRINT_STR
5      LEA DX, buf_user
6      CALL READ_STR          ; fn 0Ah fills buf_user
7
8      LEA DX, prompt_pass
9      CALL PRINT_STR
10     LEA DX, buf_pass
11     CALL READ_STR          ; fn 0Ah fills buf_pass
12
13     ; Compare entered password (buf_pass+2) vs "user$"
14     LEA SI, buf_pass + 2    ; SI -> actual characters
15     LEA DI, user_pass       ; DI -> reference string
16     CALL STR_COMPARE        ; ZF=1 if passwords match
17     JNE UL_FAIL            ; Jump if not equal
18
19     LEA DX, ok_login
20     CALL PRINT_STR
21     CALL PAUSE
22     CALL USER_MENU_PROC    ; Enter user session
23     RET
24
25 UL_FAIL:
26     LEA DX, err_login
27     CALL PRINT_STR
28     CALL PAUSE
29     RET
30 USER_LOGIN ENDP

```

Listing 3.2: USER\_LOGIN Procedure

**Registers used:** SI (source pointer for compare), DI (destination pointer for compare), DX (string pointer), AH (DOS function selector).

### 3.3.3 STR\_COMPARE Utility Procedure

This is one of the most critical utility procedures. It performs a byte-by-byte comparison between the user-entered string (which is terminated by a carriage return, 0Dh) and the reference credential string (terminated by \$). It sets the zero flag (**ZF**) upon successful equality, allowing the caller to use JE/JNE instructions for branching.

```

1  STR_COMPARE  PROC
2  SC_LOOP:
3      MOV AL, [SI]                ; Load next char from input
4      MOV BL, [DI]                ; Load next char from reference
5      CMP AL, 0Dh                 ; End of input? (carriage return
6      )
7      JE  SC_END_S                ; End of reference?
8      CMP BL, '$'                 ; End of reference?
9      JE  SC_END_D                ; Characters equal?
10     JNE SC_DIFF                 ; No: strings differ
11     INC SI                       ; Advance source pointer
12     INC DI                       ; Advance dest pointer
13     JMP SC_LOOP                 ; Next character
14
15 SC_END_S:                       ; Input ended
16     CMP BL, '$'                 ; Reference also at end?
17     JE  SC_EQUAL                 ; Yes: equal
18     JMP SC_DIFF                 ; No: input shorter -> differ
19
20 SC_END_D:                       ; Reference ended
21     CMP AL, 0Dh                 ; Input also at end?
22     JE  SC_EQUAL                 ; Yes: equal
23
24 SC_DIFF:
25     OR  AL, 1                   ; Clear ZF (unequal)
26     RET
27 SC_EQUAL:
28     XOR AL, AL                  ; Set ZF=1 (equal)
29     RET
30 STR_COMPARE  ENDP

```

Listing 3.3: STR\_COMPARE Procedure

**Flags affected:** Zero Flag (ZF). **Registers:** AL, BL, SI, DI.

### 3.3.4 USER\_MENU\_PROC Procedure

After successful user login, USER\_MENU\_PROC presents and processes the user menu in a loop, dispatching to category browsing, book search, or purchase modules based on single-character input.

```

1  USER_MENU_PROC PROC
2  UM_LOOP:
3      CALL CLEAR_SCREEN
4      LEA DX, user_menu
5      CALL PRINT_STR
6      CALL READ_CHAR          ; AL = choice
7
8      CMP AL, '1'
9      JE  UM_CAT              ; Categories
10     CMP AL, '2'
11     JE  UM_SEARCH           ; Search
12     CMP AL, '3'
13     JE  UM_BUY              ; Buy
14     CMP AL, '4'
15     JE  UM_OUT              ; Logout
16
17     LEA DX, err_choice
18     CALL PRINT_STR
19     CALL PAUSE
20     JMP UM_LOOP
21
22  UM_CAT:    CALL CATEGORY_MENU
23             JMP UM_LOOP
24  UM_SEARCH: CALL SEARCH_BOOK
25             JMP UM_LOOP
26  UM_BUY:    CALL BUY_BOOK
27             JMP UM_LOOP
28  UM_OUT:    RET
29  USER_MENU_PROC ENDP

```

Listing 3.4: USER\_MENU\_PROC – Loop and Dispatch

### 3.3.5 CATEGORY\_MENU Procedure

CATEGORY\_MENU implements a three-category menu loop. When a category is selected, the relevant book strings (stored in the data segment) are printed sequentially using

PRINT\_STR, then the loop resumes awaiting the next category choice or a back request.

```

1 CM_PROG:
2     LEA DX, cat_prog_hdr
3     CALL PRINT_STR           ; Print category header
4     LEA DX, book1
5     CALL PRINT_STR           ; Print book 1
6     LEA DX, book2
7     CALL PRINT_STR           ; Print book 2
8     CALL PAUSE
9     JMP CM_LOOP

```

Listing 3.5: CATEGORY\_MENU – Programming Category Branch

### 3.3.6 SEARCH\_BOOK Procedure

The search procedure accepts a single character representing the book serial number (1–6). It validates the input range using CMP and the JB/JA conditional jump pair (unsigned less-than / greater-than), then calls PRINT\_BOOK\_BY\_AL to display the matching book entry, or prints a “not found” message.

```

1 SEARCH_BOOK PROC
2     CALL CLEAR_SCREEN
3     LEA DX, search_msg
4     CALL PRINT_STR
5     CALL READ_CHAR           ; serial digit -> AL
6
7     CMP AL, '1'
8     JB  SB_NF                 ; Below '1' -> not found
9     CMP AL, '6'
10    JA  SB_NF                 ; Above '6' -> not found
11
12    LEA DX, search_found      ; Valid serial
13    CALL PRINT_STR
14    CALL PRINT_BOOK_BY_AL     ; Display the book
15    JMP SB_END
16
17 SB_NF:
18    LEA DX, search_nf
19    CALL PRINT_STR
20 SB_END:
21    CALL PAUSE
22    RET

```

23 SEARCH\_BOOK ENDP

Listing 3.6: SEARCH\_BOOK – Range Validation and Dispatch

**Conditional jumps used:** JB (jump if below, unsigned), JA (jump if above, unsigned). These use the carry and zero flags set by the preceding CMP instruction.

### 3.3.7 BUY\_BOOK Procedure and Billing Logic

BUY\_BOOK is the most computationally intensive procedure. It:

1. Accepts a book serial number character.
2. Validates it within range 1–6.
3. Accepts a quantity character, validated within range 1–9.
4. Computes the zero-based array index: `index = (serial_char - '1') * 2`.
5. Loads the unit price from the `prices` word array using register-indirect addressing.
6. Converts the quantity character to a numeric value.
7. Performs unsigned 16-bit multiplication (MUL CX), yielding the total in AX.
8. Prints a formatted bill receipt using a combination of string output and the PRINT\_NUMBER procedure.

```

1      ; AL = serial character ('1'..'6')
2      MOV AL, BL                ; Restore serial char
3      SUB AL, '1'               ; Convert to 0-based index
      (0..5)
4      MOV AH, 0                 ; Zero-extend to 16-bit
5      SHL AX, 1                 ; Multiply index by 2 (word size)
      )
6      MOV SI, AX                ; SI = byte offset into prices[]
7      MOV AX, prices[SI]        ; AX = unit price
8
9      MOV CL, BH                ; CL = quantity character
10     SUB CL, '0'               ; Convert ASCII to numeric
11     MOV CH, 0                 ; Zero-extend to 16-bit
12     MUL CX                    ; DX:AX = price * quantity
13     MOV total, AX             ; Store total (low word)

```

Listing 3.7: BUY\_BOOK – Array Indexing and Multiplication

**Instructions used:** SUB (ASCII digit conversion), SHL (word index scaling), MUL (unsigned multiply), MOV with base-register indirect addressing (prices[SI]).

### 3.3.8 PRINT\_NUMBER Procedure

Printing integers in assembly requires manual decimal conversion because there is no native integer-to-string function in DOS. PRINT\_NUMBER implements the standard repeated-division algorithm:

1. Divide AX by 10; remainder (DX) is the least-significant digit.
2. Push the remainder onto the stack.
3. Repeat until AX = 0, counting digits in CX.
4. Pop digits in LIFO order, add '0' (ASCII offset 48), and output via INT 21H / 02H.

```
1 PRINT_NUMBER PROC
2     PUSH AX
3     PUSH BX
4     PUSH CX
5     PUSH DX
6
7     MOV CX, 0           ; Digit counter
8     MOV BX, 10          ; Divisor
9
10    PN_DIV:
11        MOV DX, 0        ; Zero-extend AX to DX:AX
12        DIV BX            ; AX = quotient, DX = remainder
13        PUSH DX          ; Push digit (in reverse order)
14        INC CX            ; Count digits
15        CMP AX, 0
16        JNE PN_DIV       ; Continue if quotient != 0
17
18    PN_OUT:
19        POP DX            ; Pop digit (least-sig first)
20        ADD DL, '0'       ; ASCII conversion
21        MOV AH, 02h
22        INT 21h           ; Output character
23        LOOP PN_OUT       ; Repeat for all digits
24
25        POP DX
26        POP CX
```

```

27     POP BX
28     POP AX
29     RET
30 PRINT_NUMBER ENDP

```

Listing 3.8: PRINT\_NUMBER – Decimal Conversion Algorithm

**Stack usage:** Each digit is pushed as a word; for a 4-digit number (e.g., 1800 Tk), 4 words (8 bytes) are temporarily used on the stack in addition to the 4 saved registers (8 bytes) and the return address (2 bytes).

## 3.4 Register Usage Summary

Table 3.3: Summary of Register Usage Across Procedures

| Register     | Primary Usage in This Project                                                                                                               |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| AX / AL / AH | Menu input character (AL), DOS function selector (AH), arithmetic operand and quotient (AX), segment initialisation.                        |
| BX / BL / BH | Divisor in PRINT_NUMBER (BX), book serial character storage in BUY_BOOK (BL), quantity character (BH), byte comparison in STR_COMPARE (BL). |
| CX / CL      | Digit counter in PRINT_NUMBER, quantity numeric value in BUY_BOOK, loop counter for LOOP instruction.                                       |
| DX / DL      | String pointer for INT 21H / 09H (DX), output character for INT 21H / 02H (DL), division remainder in PRINT_NUMBER (DX).                    |
| SI           | Source index for STR_COMPARE; byte-offset index into <code>prices</code> array.                                                             |
| DI           | Destination index (reference string pointer) in STR_COMPARE.                                                                                |
| DS           | Data segment base register; initialised once at program entry.                                                                              |
| SP / Stack   | Managed automatically by CALL/RET; explicitly used for register preservation (PUSH/POP) in PRINT_NUMBER.                                    |

## 3.5 Error Handling

The application implements two categories of error handling:

1. **Invalid Menu Choice:** Any character outside the defined option range causes the `err_choice` message to be displayed, followed by a pause, and the menu loop

restarts.

2. **Invalid Login:** If the entered password does not match the stored credential (as determined by `STR_COMPARE`), the `err_login` message is displayed and the procedure returns to the main menu, denying access.
3. **Invalid Book Serial / Quantity:** In `SEARCH_BOOK` and `BUY_BOOK`, range validation using `JB/JA` guards against out-of-range serial numbers or quantities.

There is no interrupt-based exception handling (as the 8086 does not implement structured exceptions); all error handling is explicit conditional branching.

## 4. CCA Mapping

### 4.1 CCA1 – Complex Engineering Problem Solving

This project directly addresses **CCA1: Ability to identify, formulate, and solve complex engineering problems**, as defined in the programme outcomes for CEN 323. The core complexity of the problem lies in the fundamental mismatch between the high-level functional requirements of a retail management system (multi-user access, categorised inventory, search, and billing) and the primitive computational model available in 8086 assembly language.

At the assembly level, there are no objects, no data structures, no built-in string handling, and no floating-point arithmetic. Every feature required the team to decompose a high-level concept into a sequence of register operations, memory accesses, and conditional jumps. For example, implementing the login authentication system required understanding that the DOS 0Ah input function stores the entered string starting at byte offset 2 of the input buffer, that the string is terminated by a carriage return (0Dh) rather than a null byte, and that comparison must proceed character-by-character using pointer registers (SI, DI) with flag-based decision-making. The `STR_COMPARE` procedure was the solution: a custom-built string comparison routine that correctly handles the asymmetric termination conventions (0Dh for input, \$ for stored data) and communicates its result through the Zero Flag, allowing callers to use idiomatic `JE/JNE` branching.

The billing system presented a second complex problem: computing a monetary total and displaying it in decimal. The 8086 `MUL` instruction performs unsigned 16-bit multiplication, with the result in the `DX:AX` register pair. Because all prices and quantities in this system fit within 16 bits, only the `AX` component of the result is needed. However, the result must then be converted to a decimal ASCII string for display, a task requiring a digit-extraction loop using repeated division by 10, a stack-based digit reversal, and character-by-character output – a non-trivial algorithmic chain implemented entirely in assembly.

The array indexing challenge further illustrates problem complexity. The `prices` array stores word (16-bit) values, so a zero-based index derived from the serial character must

be scaled by 2 (`SHL AX, 1`) to yield the correct byte offset. A single misunderstanding of this word-offset requirement would cause the billing system to read the wrong price silently, a subtle bug that requires architectural awareness to prevent.

## 4.2 CCA4 – Engineering Design Under Constraints

**CCA4: Ability to design a system, component, or process meeting desired needs within realistic constraints** is extensively exercised throughout this project. The design constraints imposed by the 8086 architecture and the DOS environment are numerous and non-trivial:

**Register Scarcity:** The 8086 provides only four general-purpose 16-bit registers (`AX`, `BX`, `CX`, `DX`) and two index registers (`SI`, `DI`). In procedures such as `BUY_BOOK`, multiple pieces of state (serial character, quantity character, array index, unit price, total) must coexist, requiring careful allocation of register roles. The serial character is preserved in `BL` and the quantity in `BH`, exploiting the high/low byte split of `BX` to pack two values into one register, a design decision driven purely by register scarcity.

**Segmented Memory Addressing:** The 8086's segmented memory model means that data and code reside in different logical segments. Every data access implicitly uses `DS` as the segment base. The design decision to use `.MODEL SMALL` ensures that the entire data segment fits within 64 KB, making segment arithmetic unnecessary in application code. Had the inventory been larger, inter-segment addressing (`ES:` overrides) would have been required, substantially increasing code complexity.

**No Dynamic Memory Allocation:** The 8086 has no `malloc` or `new`. All buffers must be statically declared in the data segment. This constrained the input buffer design to fixed sizes (10 bytes per credential), limiting password length. It also means the book inventory cannot be expanded at runtime – a fundamental architectural constraint that motivates the static `prices` array and pre-defined book string layout.

**16-bit Arithmetic Limits:** All arithmetic is 16-bit. Prices and quantities were designed to ensure that the maximum possible total ( $450 \text{ Tk} \times 9 = 4050 \text{ Tk}$ ) remains within the 16-bit unsigned range (0–65535), avoiding `DX:AX` overflow handling. This is an explicit design constraint that would need to be revisited if prices or quantities were to increase significantly.

**DOS I/O Constraint:** The only console I/O mechanism available is through `INT 21H` service functions. String output requires the string to be `$`-terminated and pointed

to by `DS:DX`; character input returns a single byte in `AL`. These constraints drove the entire menu navigation design toward single-character input rather than buffered command parsing, a valid and intentional design decision within the given constraints.

## 4.3 CCA8 – Individual and Team Collaboration

**CCA8: Understanding of professional and ethical responsibility and the ability to function effectively in a team** was directly practised throughout this project’s development lifecycle. The team adopted a module-based division of labour aligned with the procedure structure of the codebase, which naturally supports parallel development in a single assembly source file using clear procedure boundaries as collaboration interfaces.

Each team member took ownership of one or more modules (detailed in Chapter 6), with defined inputs and outputs at the procedure level serving as the interface contract. For example, the utility procedure developer specified that `PRINT_STR` expects `DS:DX` to point to a `$`-terminated string and that `STR_COMPARE` returns its result via the Zero Flag – conventions that module developers could depend upon without needing visibility into the implementation.

Collaborative debugging was a significant component of the teamwork. Identifying that `DS` must be initialised before any data segment access, that `buf_pass + 2` is the correct offset for actual input characters (not `buf_pass + 1`), and that `XOR AL, AL` correctly sets ZF while `OR AL, 1` clears it – these insights were shared across the team through code review sessions using EMU8086’s step-by-step debugger. The emulator’s register panel and stack viewer served as a shared reference frame during debugging discussions.

Professionally, the project reinforced that assembly-level code requires extensive peer review: a single off-by-one error in a memory offset or a forgotten register restore can corrupt program state in ways that are difficult to diagnose without systematic inspection. The team’s commitment to code comments (every procedure and significant instruction is annotated) reflects an understanding of the documentation obligations essential to professional software engineering.

## 5. Testing & Results

### 5.1 Testing Strategy

Testing of the system was conducted directly within the EMU8086 emulator using both the step-by-step debugger and direct execution mode. Two categories of tests were performed:

- **Functional Tests:** Verifying that each feature (login, categories, search, billing, error handling) produces the correct output for valid inputs.
- **Boundary and Negative Tests:** Verifying system robustness for invalid inputs, such as out-of-range serial numbers, incorrect credentials, and boundary quantity values.

### 5.2 Test Cases

Table 5.1: Functional Test Cases and Results

| TC#  | Test Description                 | Input                                         | Expected Output                             | Result |
|------|----------------------------------|-----------------------------------------------|---------------------------------------------|--------|
| TC01 | Welcome screen display           | Program launch                                | Welcome banner printed; wait for keypress   | PASS   |
| TC02 | Main menu display                | Any key on welcome screen                     | Main menu with options 1/2/3 displayed      | PASS   |
| TC03 | User login – correct credentials | Username: <b>user</b> , Password: <b>user</b> | “Login Successful” message; user menu shown | PASS   |

| TC#  | Test Description                  | Input                            | Expected Output                                         | Result      |
|------|-----------------------------------|----------------------------------|---------------------------------------------------------|-------------|
| TC04 | User login – wrong password       | Username: user, Password: wrong  | “Wrong Username or Password” error; return to main menu | <b>PASS</b> |
| TC05 | Admin login – correct credentials | Username: admin, Password: admin | “Login Successful”; full inventory displayed            | <b>PASS</b> |
| TC06 | Admin login – wrong password      | Password: xyz                    | Error message; return to main menu                      | <b>PASS</b> |
| TC07 | View Programming books            | User menu: 1, Category: 1        | Programming category header + Book1 + Book2             | <b>PASS</b> |
| TC08 | View Fiction books                | User menu: 1, Category: 2        | Fiction header + Book3 + Book4                          | <b>PASS</b> |
| TC09 | View Science books                | User menu: 1, Category: 3        | Science header + Book5 + Book6                          | <b>PASS</b> |
| TC10 | Search valid book serial          | User menu: 2, Serial: 3          | “Book Found!” + Book3 (Harry Potter)                    | <b>PASS</b> |
| TC11 | Search invalid serial (0)         | Serial: 0                        | “Book NOT Found!” message                               | <b>PASS</b> |
| TC12 | Search invalid serial (7)         | Serial: 7                        | “Book NOT Found!” message                               | <b>PASS</b> |
| TC13 | Buy book – valid serial and qty   | Serial: 1, Qty: 2                | Bill: Serial 1, Qty 2, Unit 250, Total 500              | <b>PASS</b> |

| TC#  | Test Description                | Input                      | Expected Output                                | Result      |
|------|---------------------------------|----------------------------|------------------------------------------------|-------------|
| TC14 | Buy book – max qty (9)          | Serial: 5, Qty: 9          | Bill: Serial 5, Qty 9, Unit 450, Total 4050    | <b>PASS</b> |
| TC15 | Buy book – invalid serial (0)   | Serial: 0                  | “Invalid Choice” error message                 | <b>PASS</b> |
| TC16 | Buy book – invalid quantity (0) | Serial: 2, Qty: 0          | “Invalid Choice” error message                 | <b>PASS</b> |
| TC17 | Invalid main menu choice        | Input: 5                   | “Invalid Choice” error; main menu re-displayed | <b>PASS</b> |
| TC18 | User logout                     | User menu: 4               | Returns to main menu                           | <b>PASS</b> |
| TC19 | Program exit                    | Main menu: 3               | “Goodbye” message; program terminates          | <b>PASS</b> |
| TC20 | Billing arithmetic correctness  | Serial: 6 (350 Tk), Qty: 7 | Total = 2450 Tk                                | <b>PASS</b> |

## 5.3 Screenshots

### 5.3.1 Welcome Screen

Figure 5.1 shows the welcome interface displayed when the application starts in EMU8086.

### 5.3.2 Main Menu

Figure 5.2 illustrates the main navigation menu of the Online Book Shop Management System.

### 5.3.3 User Login

Figure 5.3 demonstrates successful user authentication after entering valid credentials.

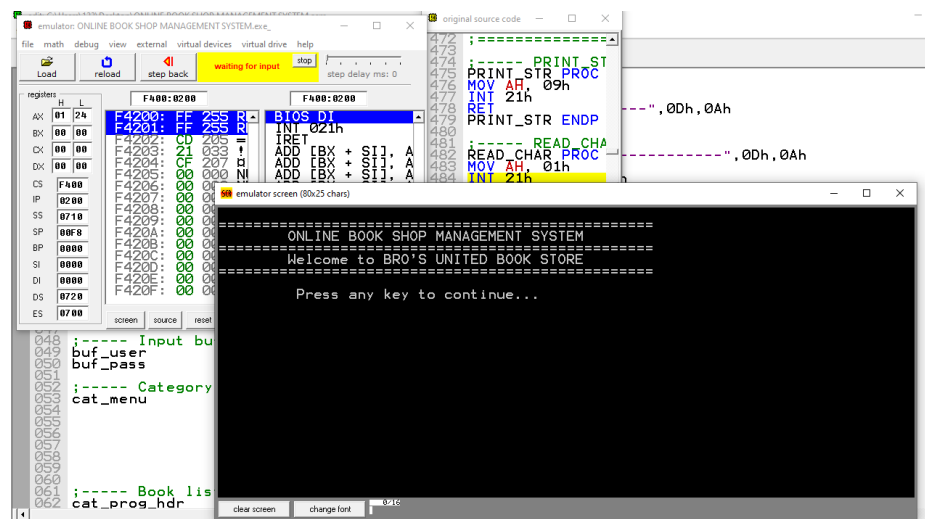


Figure 5.1: Application Welcome Screen in EMU8086

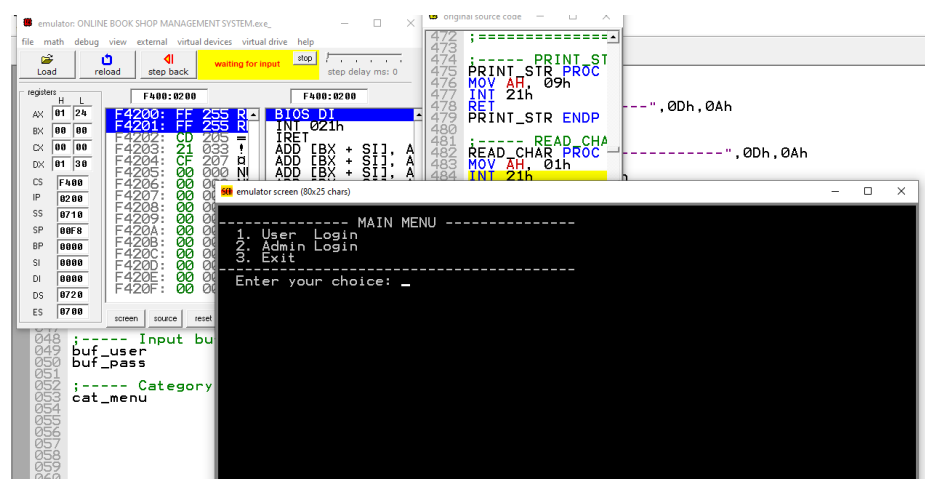


Figure 5.2: Main Menu

### 5.3.4 Book Categories

Figure 5.4 displays the Programming category containing available programming-related books.

### 5.3.5 Billing Receipt

Figure 5.5 shows a sample bill receipt generated after purchasing books from the system.

## 5.4 Bug Fixes and Observations

During testing and debugging, the following issues were encountered and resolved:

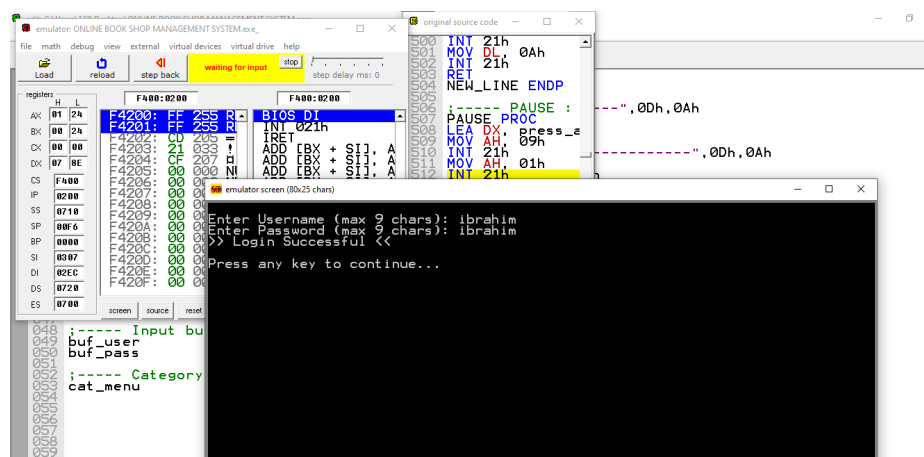


Figure 5.3: Successful User Login

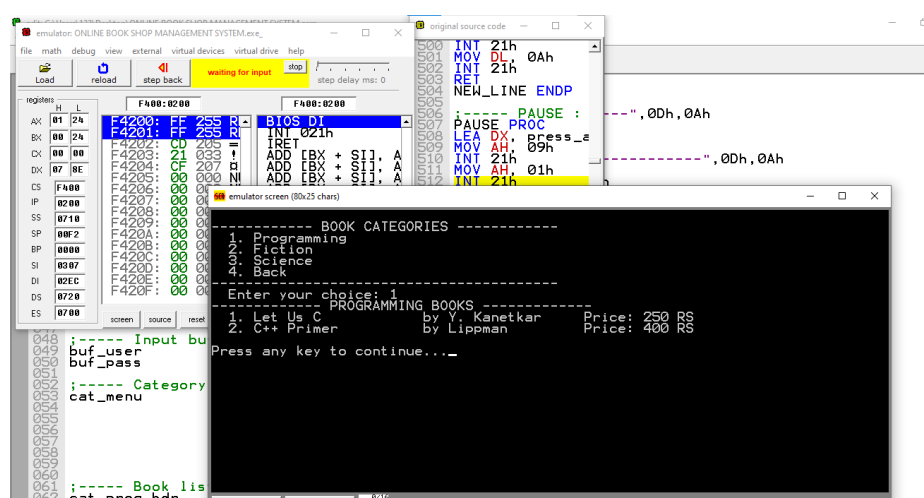


Figure 5.4: Programming Books Category View

1. **DS Not Initialised:** Initial versions failed silently because the data segment register was not initialised. Adding `MOV AX, @DATA / MOV DS, AX` at the start of `MAIN` resolved all string access failures.
2. **STR\_COMPARE Offset Error:** The first iteration of the password comparison incorrectly compared from `buf_pass + 1` (the actual-length byte) instead of `buf_pass + 2` (the first character). This caused the comparison to always fail. Correcting the `LEA SI, buf_pass + 2` offset fixed the issue.
3. **Price Array Word Offset:** An early version used `MOV SI, AX` without scaling (`SHL AX, 1`), causing the wrong price to be loaded (byte offset used as word offset). Adding `SHL AX, 1` before the array load corrected the billing values.
4. **PRINT\_NUMBER Zero Display:** For the input value 0, the initial `LOOP` check caused the number "0" to be skipped. Restructuring to increment `CX` before

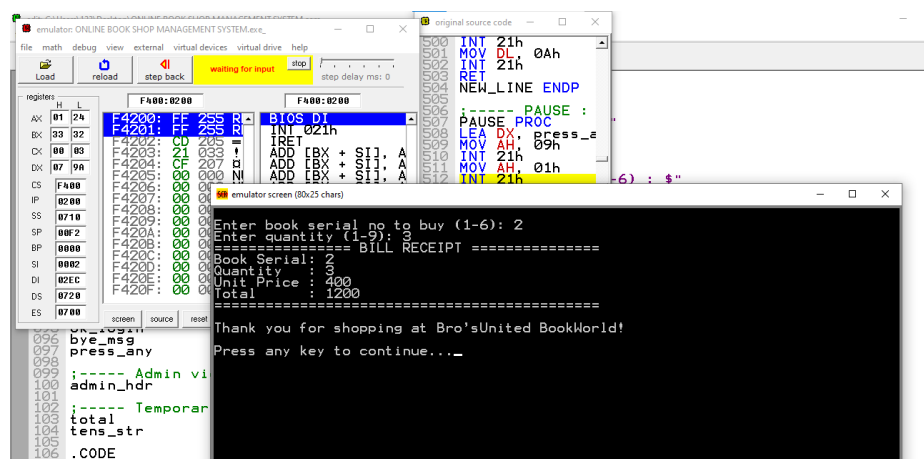


Figure 5.5: Sample Bill Receipt

the conditional jump (pushing at least one digit) resolved this edge case.

5. **Screen Not Clearing Properly:** Early use of space-filling loops left cursor artifacts. Switching to the INT 10H / 06H BIOS scroll call with a subsequent cursor reset produced clean screen clears.

## 6. Team Contributions

The project was completed collaboratively by a team of three members. Work was divided by functional module, with each member taking primary responsibility for design, implementation, and testing of their assigned module. Cross-module integration and final testing were conducted as a joint effort.

Table 6.1: Team Member Contributions

| Member Name     | Reg. No.        | Module(s) Owned                              |
|-----------------|-----------------|----------------------------------------------|
| [MUHAMMAD UMER] | [01-135232-071] | MAIN procedure, User/Admin Login, STR_COMPAR |
| [IBRAHIM SAUD]  | [01-135232-059] | USER_MENU_PROC, CATEGORY_MENU, SEARCH        |

Table 6.2: Task Distribution by Activity

| Activity                           | Member 1 | Member 2 |  |
|------------------------------------|----------|----------|--|
| Requirement Analysis               | ✓        | ✓        |  |
| System Design                      | ✓        | ✓        |  |
| Core Logic (Login, Authentication) | ✓        |          |  |
| Menu & Category Module             |          | ✓        |  |
| Search Module                      |          | ✓        |  |
| Billing & Arithmetic Module        | ✓        |          |  |
| Utility Procedures                 | ✓        |          |  |
| Debugging & Integration Testing    | ✓        | ✓        |  |
| Report Writing                     | ✓        |          |  |
| Presentation Preparation           | ✓        | ✓        |  |

The team maintained regular communication through WhatsApp group messaging and bi-weekly lab sessions. Version control was managed via a private GitHub repository, with each member committing to their designated feature branch before merging into the main branch upon peer review.

# 7. Challenges & Lessons Learned

## 7.1 Challenges Encountered

### 7.1.1 Register Management and Data Preservation

The most pervasive challenge throughout development was the limited number of registers available in the 8086 architecture. With only four general-purpose registers, every procedure had to be carefully designed to avoid clobbering register values that calling procedures depended upon. Early versions of `BUY_BOOK` lost the serial number stored in `BL` when `PRINT_STR` modified `AH` and subsequent interrupt calls affected `AX`, corrupting the value in `AL`. The solution was to save `BL` and `BH` before any interrupt call and restore them immediately after. This experience drove the adoption of push/pop preservation discipline in all utility procedures.

### 7.1.2 Debugging at the Assembly Level

Debugging in assembly language is fundamentally different from debugging in high-level languages. There are no meaningful runtime error messages; a mistake in a memory offset or an incorrect register value manifests as corrupted output, an infinite loop, or a program crash. The team invested significant time learning to use EMU8086's integrated debugger effectively, including setting breakpoints at procedure entry points, single-stepping through critical loops (such as `STR_COMPARE`), and inspecting the memory dump to verify the exact content of input buffers after the `0AH` call.

### 7.1.3 Understanding DOS Interrupt Conventions

The DOS `INT 21H` interrupt services have strict register conventions that must be observed precisely. The team initially confused function `09H` (print string, expects `DS:DX`) with function `02H` (print character, expects `DL`), causing garbage output. Systematic referencing of the Ralf Brown Interrupt List and the Intel 8086 family manuals clarified each function's requirements. Similarly, understanding that `INT 21H / 0AH` includes a carriage return (`0Dh`) at the end of the input – not a null byte – was critical to fixing the string comparison procedure.

### 7.1.4 ASCII and Numeric Conversions

The 8086 performs arithmetic on binary values, while all input and output is in ASCII text. Converting between these representations – subtracting '0' (48) to convert a digit character to a number, and adding '0' to convert back – introduced subtle bugs when the conversion was omitted or applied at the wrong point. The billing quantity computation, for instance, required subtracting '0' from the quantity character before the MUL instruction, a step easy to overlook.

### 7.1.5 Memory Layout and Offset Arithmetic

Understanding the exact memory layout of the 0AH input buffer (byte 0 = max length, byte 1 = actual length, byte 2 onward = characters) required careful reading of DOS documentation and verification via the memory dump view. The initial offset error (+1 instead of +2) caused the comparison to read the “actual characters typed” byte instead of the first character, producing a consistently failing login regardless of input.

### 7.1.6 Screen Management Limitations

The EMU8086 virtual terminal does not support ANSI escape codes, making cursor positioning and screen formatting more challenging. The team relied on INT 10H for screen clearing and on embedded 0Dh / 0Ah sequences within string data for line formatting, requiring all layout to be pre-planned as static text rather than dynamically generated.

### 7.1.7 Team Coordination

Coordinating development of a single assembly file across three developers required explicit naming conventions for labels and procedures to avoid symbol conflicts. The team adopted a policy of prefixing local labels with a procedure abbreviation (e.g., UM\_LOOP for User Menu loop, CM\_BACK for Category Menu back) and communicating interface specifications (register contracts) before implementation. This prevented integration-time conflicts.

## 7.2 Lessons Learned

- **Plan Before Coding:** In assembly, a clear design specifying which registers hold which values at each point in a procedure is essential before writing a single

instruction.

- **Document Everything:** Assembly code is inherently opaque without comments. Liberal commenting – every procedure, every non-obvious instruction – is not a luxury but a necessity.
- **Trust the Debugger:** EMU8086's step-through debugger, register panel, and memory viewer are indispensable tools. Using them systematically (not as a last resort) dramatically reduces debugging time.
- **Understand Hardware:** This project reinforced that software is ultimately a sequence of hardware operations. Understanding instruction cycle behaviour, flag semantics, and interrupt handling mechanisms builds a mental model that is invaluable for all subsequent programming, at any abstraction level.
- **Modular Design Scales Even in Assembly:** Decomposing the application into small, single-responsibility procedures with well-defined interfaces made the codebase far more manageable than a monolithic implementation would have been.

# 8. Conclusion

## 8.1 Achievements

This project successfully delivered a fully functional, menu-driven Online Book Shop Management System implemented in 8086 Assembly Language and executed on the EMU8086 emulator. All core features specified in the project requirements were implemented and verified through systematic test cases:

- A welcome screen with keypress-to-continue navigation.
- A dual-access authentication system supporting both user and administrator roles, secured by password comparison at the assembly level.
- A category-based book browsing system supporting Programming, Fiction, and Science categories.
- A book search facility supporting serial-number-based lookup with range validation.
- A complete purchase and billing system with unit price lookup, quantity-based total computation via 16-bit multiplication, and formatted receipt output.
- Robust error handling for all invalid inputs across menus, login, search, and purchase modules.
- A clean administrative inventory view displaying the complete book catalogue.

## 8.2 Learning Outcomes

Through the implementation of this project, the team achieved the following learning outcomes:

- Mastery of the 8086 segmented memory model and the correct use of the data, code, and stack segments within the `.MODEL SMALL` framework.
- Practical application of DOS `INT 21H` services for all console I/O, including single-character input, string output, buffered keyboard input, and program

termination.

- Design and implementation of reusable utility procedures following strict register preservation conventions.
- Implementation of a custom string comparison algorithm adapted to the mixed termination conventions of DOS input buffers.
- Understanding of 16-bit unsigned arithmetic including the `MUL` instruction and the digit-extraction algorithm required for decimal output without high-level library support.
- Experience with assembly-level debugging using EMU8086's integrated debugging tools.
- Practice in module-based collaborative software development with version control.

## 8.3 Limitations

The current implementation has the following acknowledged limitations:

- **Hard-coded Credentials and Inventory:** There is no file I/O; all data is static. Credentials cannot be changed and inventory cannot be updated without modifying and reassembling the source code.
- **Single-item Billing:** The billing system supports purchasing one book type per transaction. A shopping cart supporting multiple different books in a single session is not implemented.
- **No Case Insensitivity:** Login passwords are case-sensitive. There is no input normalisation.
- **Limited Quantity Range:** Quantities are restricted to 1–9 (single digit), an architectural simplification for single-character input.
- **No Persistent Storage:** Purchase records are not saved; there is no transaction history.

## 8.4 Future Improvements

The following enhancements could be implemented in future iterations:

1. **File-Based Data Storage:** Using DOS file services (INT 21H functions 3CH–3EH) to read inventory and credentials from text files, enabling runtime configuration.
2. **Multi-Item Cart System:** Implementing a small array-based shopping cart allowing users to add multiple books before generating a consolidated bill.
3. **Stock Management for Admin:** Allowing the administrator to update book quantities and prices at runtime, with changes reflected in subsequent user transactions.
4. **Password Masking:** Using INT 21H / 08H (input without echo) for password entry to prevent on-screen display of sensitive credentials.
5. **Extended Inventory:** Expanding beyond 6 books, potentially with a dynamic page-based browsing system to handle larger catalogues within the 64 KB data segment limit.
6. **Colour Output:** Leveraging INT 10H attribute bytes for coloured text to improve the user interface aesthetics.

# References

- [1] Intel Corporation, *iAPX 86/88, 186/188 User's Manual: Programmer's Reference*. Santa Clara, CA: Intel Corporation, 1983. [Online]. Available: [https://archive.org/details/bitsavers\\_inteliAPX86](https://archive.org/details/bitsavers_inteliAPX86)
- [2] R. Brown, *Ralf Brown's Interrupt List*, Release 61, 2000. [Online]. Available: <http://www.cs.cmu.edu/~ralf/files.html>
- [3] EMU8086 Development Team, *EMU8086 – Microprocessor Emulator User Manual*, v4.08. [Online]. Available: <http://www.emu8086.com>
- [4] K. Irvine, *Assembly Language for x86 Processors*, 7th ed. Hoboken, NJ: Pearson Education, 2015.
- [5] R. Dandamudi, *Fundamentals of Computer Organization and Design*. New York, NY: Springer, 2003.
- [6] P. Carter, *PC Assembly Language*, Free e-book, 2006. [Online]. Available: <http://pacman128.github.io/pcasm/>
- [7] Microsoft Corporation, *MS-DOS Programmer's Reference*, version 6.0. Redmond, WA: Microsoft Press, 1993.
- [8] [MUHAMMAD UMER], [IBRAHIM SAUD], “Online Book Shop Management System – CEN323 Project,” GitHub Repository, 20XX. [Online]. Available: <https://github.com/uq1534-cell/Online-book-shop-in-Assembly-language> [Accessed: 17 05 2026].

# A. Complete Assembly Source Code

The complete, annotated 8086 Assembly Language source code for the Online Book Shop Management System is provided below. The file is designed to be assembled and executed directly in EMU8086 v4.08 or later without modification.

```
1  ;=====
2  ;  ONLINE BOOK SHOP MANAGEMENT SYSTEM
3  ;  Platform : EMU8086 (8086 Assembly, DOS INT 21h)
4  ;  Course   : CEN 323 -- Computer Organization & Assembly
5  ;             Language
6  ;  Instructor: Adnan Jelani
7  ;=====
8  .MODEL SMALL
9  .STACK 100H
10
11 ;=====
12 ;                               DATA SEGMENT
13 ;=====
14 .DATA
15
16 ;----- Welcome Screen -----
17 welcome_msg db 0Dh,0Ah,"
18             =====",0Dh,0Ah
19             db "      ONLINE BOOK SHOP MANAGEMENT SYSTEM      ",0
20             db "-----",0Dh,0Ah
21             db "      Welcome to BRO'S UNITED BOOK STORE      "
22             db ",0Dh,0Ah
23             db "-----",0
24             db "      Press any key to continue...$"
25
26 ;----- Main Menu -----
27 main_menu db 0Dh,0Ah,"----- MAIN MENU -----",0Dh
28           ,0Ah
29           db "  1. User  Login",0Dh,0Ah
30           db "  2. Admin Login",0Dh,0Ah
```

```

28         db " 3. Exit",0Dh,0Ah
29         db "-----",0Dh,0Ah
30         db " Enter your choice : $"
31
32 ;----- User Menu -----
33 user_menu db 0Dh,0Ah,"----- USER MENU -----",0Dh,
           ,0Ah
34         db " 1. View Book Categories",0Dh,0Ah
35         db " 2. Search Book by Serial No.",0Dh,0Ah
36         db " 3. Buy Book (Cart & Billing)",0Dh,0Ah
37         db " 4. Logout",0Dh,0Ah
38         db "-----",0Dh,0Ah
39         db " Enter your choice : $"
40
41 ;----- Login Prompts -----
42 prompt_user db 0Dh,0Ah,"Enter Username (max 9 chars) : $"
43 prompt_pass db 0Dh,0Ah,"Enter Password (max 9 chars) : $"
44
45 ;----- Hard-coded credentials -----
46 user_pass db "ibrahim$" ; User password
47 admin_pass db "umer$" ; Admin password
48
49 ;----- Input buffers (DOS function 0Ah) -----
50 buf_user db 10, 0, 10 dup('$')
51 buf_pass db 10, 0, 10 dup('$')
52
53 ;----- Category Menu -----
54 cat_menu db 0Dh,0Ah,"----- BOOK CATEGORIES -----",0Dh,0
           Ah
55         db " 1. Programming",0Dh,0Ah
56         db " 2. Fiction",0Dh,0Ah
57         db " 3. Science",0Dh,0Ah
58         db " 4. Back",0Dh,0Ah
59         db "-----",0Dh,0Ah
60         db " Enter your choice : $"
61
62 ;----- Category headers -----
63 cat_prog_hdr db 0Dh,0Ah,"----- PROGRAMMING BOOKS -----",0
           Dh,0Ah,"$"
64 cat_fic_hdr db 0Dh,0Ah,"----- FICTION BOOKS -----"
           ,0Dh,0Ah,"$"
65 cat_sci_hdr db 0Dh,0Ah,"----- SCIENCE BOOKS -----"
           ,0Dh,0Ah,"$"

```

```

66
67 ;----- Book entries -----
68 book1 db " 1. Let Us C by Y. Kanetkar Price: 250
        Tk",0Dh,0Ah,"$"
69 book2 db " 2. C++ Primer by Lippman Price: 400
        Tk",0Dh,0Ah,"$"
70 book3 db " 3. Harry Potter by J.K. Rowling Price: 500
        Tk",0Dh,0Ah,"$"
71 book4 db " 4. The Alchemist by Paulo Coelho Price: 300
        Tk",0Dh,0Ah,"$"
72 book5 db " 5. A Brief History by S. Hawking Price: 450
        Tk",0Dh,0Ah,"$"
73 book6 db " 6. Cosmos by Carl Sagan Price: 350
        Tk",0Dh,0Ah,"$"
74
75 ;----- Parallel prices array -----
76 prices dw 250, 400, 500, 300, 450, 350
77
78 ;----- Search messages -----
79 search_msg db 0Dh,0Ah,"Enter book serial no (1-6) : $"
80 search_found db 0Dh,0Ah,"Book Found!",0Dh,0Ah,"$"
81 search_nf db 0Dh,0Ah,"Book NOT Found!",0Dh,0Ah,"$"
82
83 ;----- Billing messages -----
84 buy_msg db 0Dh,0Ah,"Enter book serial no to buy (1-6) : $"
85 qty_msg db 0Dh,0Ah,"Enter quantity (1-9) : $"
86 bill_hdr db 0Dh,0Ah,"===== BILL RECEIPT =====",0Dh,0Ah,"$"
87 bill_book db "Book Serial : $"
88 bill_qty db 0Dh,0Ah,"Quantity : $"
89 bill_unit db 0Dh,0Ah,"Unit Price : $"
90 bill_total db 0Dh,0Ah,"Total : $"
91 bill_end db 0Dh,0Ah,"===== ",0Dh,0Ah,"$"
92 thank_msg db 0Dh,0Ah,"Thank you for shopping at BookWorld!",0Dh,0
        Ah,"$"
93
94 ;----- Status / error messages -----
95 err_choice db 0Dh,0Ah,"!! Invalid Choice. Try again. !!",0Dh,0Ah,
        "$"
96 err_login db 0Dh,0Ah,"!! Wrong Username or Password !!",0Dh,0Ah,
        "$"
97 ok_login db 0Dh,0Ah,">> Login Successful <<",0Dh,0Ah,"$"
98 bye_msg db 0Dh,0Ah,"Goodbye! Program terminated.",0Dh,0Ah,"$"
99 press_any db 0Dh,0Ah,"Press any key to continue...$"

```

```

100
101 ;----- Admin header -----
102 admin_hdr    db 0Dh,0Ah,"==== FULL INVENTORY (ADMIN) =====",0Dh,0
103              Ah,"$"
104
105 ;----- Shared variables -----
106
107 total dw 0
108
109 .CODE
110 ;=====
111 ;                               MAIN PROCEDURE
112 ;=====
113 MAIN PROC
114     MOV AX, @DATA                ; Initialise data segment
115     MOV DS, AX
116
117     CALL CLEAR_SCREEN            ; Show welcome screen
118     LEA DX, welcome_msg
119     CALL PRINT_STR
120     CALL READ_CHAR                ; Wait for any key
121
122 MAIN_LOOP:
123     CALL CLEAR_SCREEN
124     LEA DX, main_menu
125     CALL PRINT_STR
126     CALL READ_CHAR                ; Read menu choice into AL
127
128     CMP AL, '1'
129     JE GO_USER
130     CMP AL, '2'
131     JE GO_ADMIN
132     CMP AL, '3'
133     JE GO_EXIT
134
135     LEA DX, err_choice            ; Invalid choice
136     CALL PRINT_STR
137     CALL PAUSE
138     JMP MAIN_LOOP
139
140 GO_USER:
141     CALL USER_LOGIN
142     JMP MAIN_LOOP
143
144 GO_ADMIN:

```

```

142         CALL ADMIN_LOGIN
143         JMP MAIN_LOOP
144 GO_EXIT:
145         LEA DX, bye_msg
146         CALL PRINT_STR
147         MOV AH, 4Ch           ; Terminate program
148         INT 21h
149 MAIN ENDP
150
151 ;=====
152 ;                USER LOGIN
153 ;=====
154 USER_LOGIN PROC
155         CALL CLEAR_SCREEN
156         LEA DX, prompt_user
157         CALL PRINT_STR
158         LEA DX, buf_user
159         CALL READ_STR
160
161         LEA DX, prompt_pass
162         CALL PRINT_STR
163         LEA DX, buf_pass
164         CALL READ_STR
165
166         LEA SI, buf_pass + 2   ; Point to actual password chars
167         LEA DI, user_pass      ; Point to reference string
168         CALL STR_COMPARE      ; ZF=1 if match
169         JNE UL_FAIL
170
171         LEA DX, ok_login
172         CALL PRINT_STR
173         CALL PAUSE
174         CALL USER_MENU_PROC
175         RET
176 UL_FAIL:
177         LEA DX, err_login
178         CALL PRINT_STR
179         CALL PAUSE
180         RET
181 USER_LOGIN ENDP
182
183 ;=====
184 ;                ADMIN LOGIN

```

```
185 ; =====
186 ADMIN_LOGIN PROC
187     CALL CLEAR_SCREEN
188     LEA DX, prompt_user
189     CALL PRINT_STR
190     LEA DX, buf_user
191     CALL READ_STR
192
193     LEA DX, prompt_pass
194     CALL PRINT_STR
195     LEA DX, buf_pass
196     CALL READ_STR
197
198     LEA SI, buf_pass + 2
199     LEA DI, admin_pass
200     CALL STR_COMPARE
201     JNE AL_FAIL
202
203     LEA DX, ok_login
204     CALL PRINT_STR
205     CALL SHOW_ALL_BOOKS
206     CALL PAUSE
207     RET
208 AL_FAIL:
209     LEA DX, err_login
210     CALL PRINT_STR
211     CALL PAUSE
212     RET
213 ADMIN_LOGIN ENDP
214
215 ; =====
216 ;                USER MENU
217 ; =====
218 USER_MENU_PROC PROC
219 UM_LOOP:
220     CALL CLEAR_SCREEN
221     LEA DX, user_menu
222     CALL PRINT_STR
223     CALL READ_CHAR
224
225     CMP AL, '1'
226     JE UM_CAT
227     CMP AL, '2'
```

```

228         JE UM_SEARCH
229         CMP AL, '3'
230         JE UM_BUY
231         CMP AL, '4'
232         JE UM_OUT
233
234         LEA DX, err_choice
235         CALL PRINT_STR
236         CALL PAUSE
237         JMP UM_LOOP
238
239 UM_CAT:   CALL CATEGORY_MENU
240         JMP UM_LOOP
241 UM_SEARCH: CALL SEARCH_BOOK
242         JMP UM_LOOP
243 UM_BUY:   CALL BUY_BOOK
244         JMP UM_LOOP
245 UM_OUT:   RET
246 USER_MENU_PROC ENDP
247
248 ; =====
249 ;             CATEGORY MENU
250 ; =====
251 CATEGORY_MENU PROC
252 CM_LOOP:
253         CALL CLEAR_SCREEN
254         LEA DX, cat_menu
255         CALL PRINT_STR
256         CALL READ_CHAR
257
258         CMP AL, '1'
259         JE CM_PROG
260         CMP AL, '2'
261         JE CM_FIC
262         CMP AL, '3'
263         JE CM_SCI
264         CMP AL, '4'
265         JE CM_BACK
266
267         LEA DX, err_choice
268         CALL PRINT_STR
269         CALL PAUSE
270         JMP CM_LOOP

```

```
271
272 CM_PROG:
273     LEA DX, cat_prog_hdr
274     CALL PRINT_STR
275     LEA DX, book1
276     CALL PRINT_STR
277     LEA DX, book2
278     CALL PRINT_STR
279     CALL PAUSE
280     JMP CM_LOOP
281
282 CM_FIC:
283     LEA DX, cat_fic_hdr
284     CALL PRINT_STR
285     LEA DX, book3
286     CALL PRINT_STR
287     LEA DX, book4
288     CALL PRINT_STR
289     CALL PAUSE
290     JMP CM_LOOP
291
292 CM_SCI:
293     LEA DX, cat_sci_hdr
294     CALL PRINT_STR
295     LEA DX, book5
296     CALL PRINT_STR
297     LEA DX, book6
298     CALL PRINT_STR
299     CALL PAUSE
300     JMP CM_LOOP
301
302 CM_BACK:
303     RET
304
305 CATEGORY_MENU ENDP
306
307 ;=====
308 ;                               SHOW ALL BOOKS (Admin)
309 ;=====
310
311 SHOW_ALL_BOOKS PROC
312     LEA DX, admin_hdr
313     CALL PRINT_STR
314     LEA DX, book1
315     CALL PRINT_STR
316     LEA DX, book2
317     CALL PRINT_STR
318     LEA DX, book3
```

```

314         CALL PRINT_STR
315         LEA DX, book4
316         CALL PRINT_STR
317         LEA DX, book5
318         CALL PRINT_STR
319         LEA DX, book6
320         CALL PRINT_STR
321         RET
322 SHOW_ALL_BOOKS ENDP
323
324 ;=====
325 ;                      SEARCH BOOK
326 ;=====
327 SEARCH_BOOK PROC
328         CALL CLEAR_SCREEN
329         LEA DX, search_msg
330         CALL PRINT_STR
331         CALL READ_CHAR          ; Serial digit -> AL
332
333         CMP AL, '1'
334         JB  SB_NF              ; < '1': not found
335         CMP AL, '6'
336         JA  SB_NF              ; > '6': not found
337
338         LEA DX, search_found
339         CALL PRINT_STR
340         CALL PRINT_BOOK_BY_AL  ; Display book
341         JMP SB_END
342
343 SB_NF:
344         LEA DX, search_nf
345         CALL PRINT_STR
346 SB_END:
347         CALL PAUSE
348         RET
349 SEARCH_BOOK ENDP
350
351 ;=====
352 ;                      BUY BOOK + BILLING
353 ;=====
354 BUY_BOOK PROC
355         CALL CLEAR_SCREEN
356         LEA DX, buy_msg

```

```
357     CALL PRINT_STR
358     CALL READ_CHAR
359     MOV BL, AL                ; Save serial in BL
360
361     CMP BL, '1'
362     JB  BB_BAD
363     CMP BL, '6'
364     JA  BB_BAD
365
366     LEA DX, qty_msg
367     CALL PRINT_STR
368     CALL READ_CHAR          ; Quantity -> AL
369     CMP AL, '1'
370     JB  BB_BAD
371     CMP AL, '9'
372     JA  BB_BAD
373     MOV BH, AL              ; Save quantity in BH
374
375     ; Compute price array index
376     MOV AL, BL
377     SUB AL, '1'              ; 0-based index
378     MOV AH, 0
379     SHL AX, 1                ; Word offset (* 2)
380     MOV SI, AX
381     MOV AX, prices[SI]       ; Load unit price
382
383     ; Convert quantity to numeric
384     MOV CL, BH
385     SUB CL, '0'              ; ASCII -> numeric
386     MOV CH, 0
387     MUL CX                    ; AX = price * qty
388     MOV total, AX
389
390     ; Print bill receipt
391     LEA DX, bill_hdr
392     CALL PRINT_STR
393     LEA DX, bill_book
394     CALL PRINT_STR
395     MOV DL, BL
396     MOV AH, 02h
397     INT 21h                  ; Print serial digit
398
399     LEA DX, bill_qty
```

```

400     CALL PRINT_STR
401     MOV DL, BH
402     MOV AH, 02h
403     INT 21h                ; Print quantity digit
404
405     LEA DX, bill_unit
406     CALL PRINT_STR
407     MOV AX, prices[SI]
408     CALL PRINT_NUMBER      ; Print unit price
409
410     LEA DX, bill_total
411     CALL PRINT_STR
412     MOV AX, total
413     CALL PRINT_NUMBER      ; Print total
414
415     LEA DX, bill_end
416     CALL PRINT_STR
417     LEA DX, thank_msg
418     CALL PRINT_STR
419     CALL PAUSE
420     RET
421
422 BB_BAD:
423     LEA DX, err_choice
424     CALL PRINT_STR
425     CALL PAUSE
426     RET
427 BUY_BOOK ENDP
428
429 ;=====
430 ;          PRINT BOOK BY SERIAL NUMBER IN AL
431 ;=====
432 PRINT_BOOK_BY_AL PROC
433     CMP AL, '1'
434     JNE PB2
435     LEA DX, book1
436     JMP PB_DONE
437 PB2:    CMP AL, '2'
438     JNE PB3
439     LEA DX, book2
440     JMP PB_DONE
441 PB3:    CMP AL, '3'
442     JNE PB4

```

```

443         LEA DX, book3
444         JMP PB_DONE
445 PB4:     CMP AL, '4'
446         JNE PB5
447         LEA DX, book4
448         JMP PB_DONE
449 PB5:     CMP AL, '5'
450         JNE PB6
451         LEA DX, book5
452         JMP PB_DONE
453 PB6:     LEA DX, book6
454 PB_DONE:
455         CALL PRINT_STR
456         RET
457 PRINT_BOOK_BY_AL ENDP
458
459 ;=====
460 ;                UTILITY PROCEDURES
461 ;=====
462
463 PRINT_STR PROC                ; Print $-terminated string at DS:DX
464     MOV AH, 09h
465     INT 21h
466     RET
467 PRINT_STR ENDP
468
469 READ_CHAR PROC                ; Read 1 char with echo -> AL
470     MOV AH, 01h
471     INT 21h
472     RET
473 READ_CHAR ENDP
474
475 READ_STR PROC                ; Buffered string input at DS:DX
476     MOV AH, 0Ah
477     INT 21h
478     RET
479 READ_STR ENDP
480
481 NEW_LINE PROC                ; Output CR + LF
482     MOV AH, 02h
483     MOV DL, 0Dh
484     INT 21h
485     MOV DL, 0Ah

```

```

486         INT 21h
487         RET
488 NEW_LINE ENDP
489
490 PAUSE PROC                                ; Print "press any key" and wait
491         LEA DX, press_any
492         MOV AH, 09h
493         INT 21h
494         MOV AH, 01h
495         INT 21h
496         CALL NEW_LINE
497         RET
498 PAUSE ENDP
499
500 CLEAR_SCREEN PROC                        ; BIOS scroll up clears screen
501         MOV AX, 0600h                    ; Scroll entire window
502         MOV BH, 07h                      ; White on black attribute
503         MOV CX, 0000h                    ; Top-left corner
504         MOV DX, 184Fh                    ; Bottom-right (row 24, col 79)
505         INT 10h
506         MOV AH, 02h                      ; Reset cursor to (0,0)
507         MOV BH, 00h
508         MOV DX, 0000h
509         INT 10h
510         RET
511 CLEAR_SCREEN ENDP
512
513 STR_COMPARE PROC                        ; Compare [SI] vs [DI]; ZF=1 if equal
514 SC_LOOP:
515         MOV AL, [SI]
516         MOV BL, [DI]
517         CMP AL, 0Dh                      ; End of input?
518         JE SC_END_S
519         CMP BL, '$'                      ; End of reference?
520         JE SC_END_D
521         CMP AL, BL
522         JNE SC_DIFF
523         INC SI
524         INC DI
525         JMP SC_LOOP
526 SC_END_S:
527         CMP BL, '$'
528         JE SC_EQUAL

```

```

529             JMP SC_DIFF
530 SC_END_D:
531             CMP AL, 0Dh
532             JE SC_EQUAL
533 SC_DIFF:
534             OR AL, 1           ; Clear ZF
535             RET
536 SC_EQUAL:
537             XOR AL, AL       ; Set ZF=1
538             RET
539 STR_COMPARE ENDP
540
541 PRINT_NUMBER PROC           ; Print unsigned word in AX as decimal
542             PUSH AX
543             PUSH BX
544             PUSH CX
545             PUSH DX
546             MOV CX, 0
547             MOV BX, 10
548 PN_DIV:
549             MOV DX, 0
550             DIV BX           ; AX=quotient, DX=remainder
551             PUSH DX         ; Push digit
552             INC CX
553             CMP AX, 0
554             JNE PN_DIV
555 PN_OUT:
556             POP DX
557             ADD DL, '0'      ; ASCII conversion
558             MOV AH, 02h
559             INT 21h
560             LOOP PN_OUT
561             POP DX
562             POP CX
563             POP BX
564             POP AX
565             RET
566 PRINT_NUMBER ENDP
567
568 END MAIN

```

Listing A.1: Online Book Shop Management System – Complete Source (bookshop.asm)

## B. Glossary of Assembly Terms

Table B.1: Glossary of Key Assembly Language Terms

| Term               | Definition                                                                                                                                                |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| INT                | Software interrupt instruction; transfers control to a system service routine identified by an interrupt vector number.                                   |
| PROC / ENDP        | Directives marking the beginning and end of a named procedure in assembly.                                                                                |
| DB / DW            | Define Byte / Define Word; directives that allocate and optionally initialise memory in the data segment.                                                 |
| LEA                | Load Effective Address; computes the offset of a variable and loads it into a register (does not access memory).                                          |
| CMP                | Compare instruction; subtracts the source from the destination without storing the result, setting flags only.                                            |
| JE / JNE / JA / JB | Conditional jump instructions based on the Zero Flag and Carry/unsigned comparison flags.                                                                 |
| PUSH / POP         | Stack operations; save a register value to the stack (PUSH) or restore it (POP), following LIFO discipline.                                               |
| MUL                | Unsigned multiply; multiplies <code>AX</code> by the specified operand, storing the result in <code>DX:AX</code> .                                        |
| DIV                | Unsigned divide; divides <code>DX:AX</code> by the operand; quotient in <code>AX</code> , remainder in <code>DX</code> .                                  |
| SHL                | Shift Left; multiplies the operand by powers of 2; used here to scale an index by 2 for word addressing.                                                  |
| Zero Flag (ZF)     | A processor flag set to 1 when the result of an operation is zero; used by <code>JE/JNE</code> for branching.                                             |
| Carry Flag (CF)    | A processor flag set when an unsigned arithmetic overflow or underflow occurs; used by <code>JB/JA</code> .                                               |
| Segmented Memory   | The 8086 memory model dividing the address space into 64 KB segments addressed by <code>CS</code> , <code>DS</code> , <code>SS</code> , <code>ES</code> . |